Decision models — a possible refinement of the architecture

Status: promoted as D73. `decision-table.fs` is pinned by tests, and `DecisionTableEmitter.emit` now renders one property per row through a host `TableBinding`; the completeness check remains `DecisionTable.check`. There is still no Prose sentence surface, no parser change, and no vocabulary change. A domain opts in; a domain that never does is unaffected. The decision records are **D13** and **D73**.

This document states what a decision model is here, why it takes the shape it does, what it can and cannot do, and where it attaches now that it is promoted.

---

1. Two different things are called "decision model"

The term covers two unrelated artifacts, and conflating them produces bad architecture:

- **A decision model in the Decision Model and Notation sense**: a symbolic table of `condition → action` rows. It has no probabilities. It is a *specification*.
- **A decision model in the machine-learning sense**: a fast classifier or scorer that picks from options supplied in advance. It emits a probability or an index. It is a *policy*.

This refinement is the first kind. The second kind is deliberately excluded, and section 8 says why in terms of this project's trust boundary.

The distinction matters because the two have opposite relationships to generation. A symbolic table can be *proved complete*; a classifier cannot. A classifier can rank candidates; a table cannot invent them. Neither can write a property test from scratch, and neither is asked to.

---

2. What already existed

The architecture already had both halves of a decision model, unjoined:

Piece	Where it lives	What it does
`ScaleFile.Guard`	`scale-file.fs`	`GreaterThan`, `AtLeast`, `LessThan`, `AtMost`, `IsTrue`, `IsFalse` over a named field
`Refiner.evaluateGuard`	`refiner.fs`	decides a guard against a subject, purely
`Smt.guardToBooleanExpression`	`smt.fs`	encodes the same guard for Z3
`Smt.satisfiable`	`smt.fs`	a three-way satisfiability answer with a model, separate from `discharge`

So a decision *row* — a guard conjunction plus an outcome — was already expressible. What was missing was the **table**: the ordered collection, the hit policy, and the two checks that make a table trustworthy.

---

3. Why rows plus a completeness obligation, not a branching body

Two shapes were possible.

Shape 1 — N guarded rows plus a completeness obligation. A table is a list of rows; each row is a conjunction of guards and an outcome. The table is sound when every input selects a row (and, under `Unique`, exactly one). This is what `decision-table.fs` implements.

Shape 2 — a conditional verb body. Add `Ite(condition, then, else)` to `DomainExpression.Expression` and encode it as Z3's `MkITE`. Then one verb body *is* the whole decision.

Shape 1 was chosen, for three reasons:

1. It reuses the guards and the solver that already exist and are already property-pinned. Shape 2 would add a control construct to a fragment that D10 kept deliberately small.
2. It keeps the interesting claim **provable**. The valuable part of a decision model is not the branching; it is *no gaps and no overlaps*, and that is exactly what the existing solver can discharge rather than assert.
3. It does not require touching the parser or the vocabulary. The surface has a `Conditional` marker (`when`), but D18 refuses it with `ConditionalNotAllowed`, and `If … then …` does not parse either. Shape 1 needs none of that.

Shape 2 remains available if a verb ever needs to branch inside a single body. It is not built.

---

4. The types

```
type HitPolicy =
    | Unique      // exactly one row may match; disjointness is proved
    | First       // the first matching row wins; overlap resolved by order
    | Priority    // the matching row with the most guards wins; ties by order

type Row<'outcome> =
    { Guards: ScaleFile.Guard list
      Outcome: 'outcome }

type Table<'outcome> =
    { Name: string
      Inputs: (string * DomainExpression.FieldType) list
      Policy: HitPolicy
      Rows: Row<'outcome> list }
```

`Outcome` is a type parameter, not an `Expression`. The core does not know what an outcome *means*; that is the domain's business. This is the same discipline as `DomainBinding` (D9): the core supplies the mechanism, the domain supplies the meaning as data.

`Inputs` names every field the guards may mention, with its sort. It is what `Smt.createDomain` needs to build the theory.

---

5. The two properties

`DecisionTable.check` answers two questions. Both are Z3 queries over the integer/boolean theory the rest of the pipeline already uses.

Completeness — no gaps. For rows $g_1 \ \ldots \ g_n$:

$$\text{satisfiable}(\ \neg(g_1 \vee \ldots \vee g_n)\ )$$

A satisfiable answer is a concrete input no row covers. Reported as `UncoveredInput witness`.

Disjointness — no overlaps. Under `Unique`, for every pair `i < j`:

$$\text{satisfiable}(\ g_i \wedge g_j\ )$$

A satisfiable answer is a concrete input two rows both cover. Reported as `OverlappingRows (i, j) witness`.

The verdict is:

```
type Finding =
    | UncoveredInput of witness: Smt.Model
    | OverlappingRows of first: int * second: int * witness: Smt.Model

type Verdict =
    | Sound
    | Unsound of Finding list
    | Undecided of reason: string
```

`Sound` means every query returned `unsat`. `Undecided` means the solver gave up and is **never** read as `Sound`. An `Error` (the `Result` channel) means the table could not be encoded at all — an unknown field, or a threshold outside the integer theory. Three failure modes, kept distinct, which is the same discipline as `Outcome` in `smt.fs`.

A table with no rows is incomplete, not vacuously sound. A row with no guards is `true`, and covers everything — both are pinned by tests.

---

6. Hit policies and selection

`DecisionTable.select` is the **pure**, runtime half. It calls `Refiner.evaluateGuard`, never the solver:

```
type Selection<'outcome> =
    | Selected of index: int * row: Row<'outcome>
    | NoMatch
    | Ambiguous of indices: int list
```

Policy	What `check` requires	What `select` does on several matches
`Unique`	completeness **and** disjointness	returns `Ambiguous indices`
`First`	completeness	the first matching row
`Priority`	completeness	the matching row with the most guards; ties by row order

`Ambiguous` is deliberate: a `Unique` table that was never checked must not silently pick a row. That is the same refusal as an ill-sorted body being an `Error` rather than `false`.

`Priority` here is *structural* specificity (guard count). The refiner's coverage ranking is the stronger, solver-backed version used for candidate synthesis; the two are named differently on purpose.

---

7. The trust boundary, restated

Scale's trust boundary is narrow: a verdict is either a **proof** (`unsat` from Z3) or a **witness** (a shrunken Hedgehog counterexample). Decision tables fit because both of their properties are proofs, and both failures are witnesses.

An ML decision model does not fit that boundary. Its output is a probability. A probability is neither a proof nor a witness, so it can never be recorded as `Proved`, `Falsified`, `Sound`, or `Unsound`. It could only advise, outside the verified path, and its advice would not survive the project's determinism and offline-artifact rules. That is why the ML reading is out of scope, not merely postponed.

The same reasoning trims the usual "decision model around a test pipeline" roles: failure triage, property prioritisation, test gating, and edge-case labelling all produce *labels*, and no consumer in this repository turns a label into a verdict. With 14 deterministic properties there is also very little to triage.

8. How a domain opts in

Nothing changes for a domain that does not. A domain that wants a table writes one and calls two functions:

```
let table: DecisionTable.Table<string> =
    { Name = "jump-or-rest"
      Inputs = DemoDomain.smtFields
      Policy = DecisionTable.Unique
      Rows =
        [ { Guards = [ ScaleFile.AtLeast("health", 2.0) ]; Outcome = "jump" }
          { Guards = [ ScaleFile.AtMost("health", 1.0) ]; Outcome = "rest" } ] }

// Static: prove the table has no gaps and no overlaps.
match DecisionTable.check table with
| Ok DecisionTable.Sound -> ()
| Ok (DecisionTable.Unsound findings) -> report findings
| Ok (DecisionTable.Undecided reason) -> report reason
| Error reason -> report reason

// Dynamic: ask the table about one subject.
match DecisionTable.select DemoDomain.fieldValue table player with
| DecisionTable.Selected (index, row) -> row.Outcome
| DecisionTable.NoMatch -> ()
| DecisionTable.Ambiguous indices -> ()
```

Turning rows into property obligations

The core cannot invent a property from an outcome, but it can carry a row's guards into an obligation. The existing pieces compose without new API:

```
let obligationFor (table: DecisionTable.Table<'outcome>) (row:
DecisionTable.Row<'outcome>) =
    { Smt.Pattern = Types.Image.Capability
      Smt.Conditions = []
      Smt.Refinements =
        row.Guards
        |> List.map (fun guard ->
            { ScaleFile.Guard = guard
              Origin = "decision table " + table.Name
              Counterexample = "" }) }
```

Then, with an encoding whose `VerbBody` is the implementation under test:

```
Smt.discharge encoding (obligationFor table row)
```

checks `guards ∧ ¬body` — "on this row's inputs, the implementation agrees with the row". One obligation per row, plus the table's own completeness obligation, is the property scaffolding. The **body** still comes from the domain, because an outcome is a value and the core does not interpret it. The table supplies the structure; the domain supplies the meaning; neither pretends to be the other.

---

9. What is deliberately not built

- **No vocabulary word and no parser change.** Decision tables are a library layer, not a new sentence grammar. Adding `decide`/`otherwise` words would collide with the parser's category-based recognition (D8) and the scheduler sketch's warning against new control words. D74′s surface is a document wrapper around Prose sentence transitions, not a new sentence form.

- **The host-level binding is landed (D73).** `DecisionTable.TableBinding` supplies one body per outcome, and `DecisionTableEmitter.emit` renders one property module per row. The host supplies `row0`, `row1`, and so on.
- **No `Expression.Ite`.** Shape 2 stays unbuilt.
- **`Transition` is reachable from the surface again.** D55 re-admits the `Conditional` marker at the start of a transition sentence, so `When player is alive, every player can jump.` produces a `Transition` pattern. `emitFSharp`, `emitSmt`, and `Smt.postcondition` read it as a guarded effect, the same truth condition as `Capability` under a distinct pattern. A decision table still does not depend on `Transition`.
- **No ML inference.** See section 7.

---

10. Tests that pin it

`development/scale/tests.fs`, section "Decision tables", 15 assertions:

- a complete, disjoint `Unique` table is `Sound`;
- selection returns the row whose guards hold;
- a gap is `Unsound [UncoveredInput witness]`, and selection is `NoMatch` there;
- an overlap is `Unsound [OverlappingRows (i, j) witness]`, and selection is `Ambiguous` rather than a silent pick;
- overlap is legal under `First` and `Priority`;
- `First` resolves by order, `Priority` by guard count with an order tie-break;
- a row with no guards is `true` and covers the table;
- an empty table is incomplete;
- an unknown field is a named `Error`;
- a non-integral threshold is outside the encoded theory.

Suite total: **298 checks** (279 examples + 19 Hedgehog properties).

---

11. Where it attaches now

Three seams were already visible, and none of them required inventing a new concept:

1. **Set-theoretic typing (D12).** D12 already says a `Guard` is a comprehension and a refinement sort. A decision table is then a **partition by comprehensions**: completeness is "the union of the row comprehensions is the sort", and disjointness is "pairwise intersections are empty". The table check is the sort-level statement of the same facts. See `docs/set-theoretic-typing.typ`.
2. **The pipeline track.** Each row becomes one obligation (section 8), and the table-wide completeness check becomes one more obligation. That is a property suite derived from a specification artifact, which is the project's thesis applied one level up.
3. **`Transition`.** A decision row is the natural meaning of a transition: `when <guards> then <outcome>`. D55 admits the sentence shape and emits it as a guarded effect, so a promoted table is the multi-row semantics of the same artifact.

The promotion is D73. It is host-level: a domain declares a `Table`, a `TableBinding` with one body per outcome, and calls `DecisionTable.check` and `DecisionTableEmitter.emit`. The completeness proof remains the existing SMT check; the emitted artifact assumes the host supplies `rowN` and is not self-contained. D74 adds the Prose sentence table surface and the diagnostic sink.

12. The Prose sentence table surface (D74)

A table is not inferred from the prose specification in this slice. Its source is the **table document**:

```
table jump-or-run
policy: unique
when player has health at least 2, every player can jump.
when player has health at most 1, every player can run.
end
```

The first row's subject is the table subject; every row must name the same subject. A row's trigger conditions become its guards, and its verb becomes the outcome. Input sorts are inferred from the guards: a numeric comparison is an integer field, and a boolean condition is a boolean field. `DecisionTableSurface.parse` returns a `TableDocument`; `toTable` projects the host `Table`; `binding` supplies the subject and verb for the per-row emitter.

The table document is authored (or generated by a host); the host also supplies the domain's field getters and the outcome body. The table is therefore an abstraction of decision logic, not a hard-coded F# table and not a second sentence grammar. The demo's table text in `bases/scale-cli/entry.fs` is a hand-written example; the product path is the parsed document.

`Diagnostics.Sink` is the common sink. `add` appends a violation, `addOption` accepts an optional one, and `addSelection` adapts `DecisionTable.explain` for a `NoMatch` or `Ambiguous` selection. `render` and `renderTechnical` delegate to `Presentation`.

A disjunctive row is refused (`UnsupportedRowCondition "disjunction"`) because a table row is a conjunction of guards; a `NamedGuard` row is not produced from this surface. The host supplies the outcome body. A CLI table mode and character offsets remain TODO.